

Asset Payload Dispatch

Last updated: 2026-05-11

The payload system is the mechanism by which studios share asset data with vendors inside ArtHound. A studio selects one or more canonical assets, chooses which fields to expose, and dispatches a frozen snapshot to a specific vendor. The vendor maps those fields to their own source tool's schema and ingests. The payload system is the mechanism by which studios share asset data with vendors inside ArtHound. A studio selects one or more canonical assets, chooses which fields to expose, and dispatches a frozen snapshot to a specific vendor. The vendor maps those fields to their own source tool's schema and ingests, creating a real record in their Jira or Airtable, while ArtHound writes a permanent canonical link so the asset's journey through the vendor is traceable for the life of the production.

This is not a read-access grant. The vendor never touches the studio's source tool. The studio controls exactly what fields are shared, when access expires, and can revoke at any time.

Concepts

Template: a reusable studio-defined list of which asset fields to include in a dispatch. Each template entry carries `{key, label, type}`. Templates are optional; an untemplatised dispatch includes all fields from the asset's `meta` blob. Studios manage templates from the payload settings panel.

Dispatch: one row in `payload_dispatches` per dispatched asset. Holds a frozen `payload_data` JSONB blob (`{asset_global_id, schema, data, dispatched_at}`), a SHA-256 token hash, an expiry timestamp, and the `recipient_vendor_id`. Created atomically at dispatch time; never mutated after creation.

Snapshot: the frozen copy of the asset's field data at the moment of dispatch. Changes to the studio's source tool after dispatch do not affect what the vendor sees. This is intentional: the studio controls the version the vendor works from.

Field mapping: the vendor's saved translation of payload field keys to their own source tool field keys. Stored in `payload_field_mappings`. Keyed on `(dispatch_id, recipient_vendor_id)`; one row per dispatch per vendor.

Ingest: the act of creating a real record in the vendor's source tool from the payload data and writing a canonical link back to ArtHound. Terminal success state: `ingested_at IS NOT NULL`. Terminal failure state: `failed_at IS NOT NULL` (retryable).

Canonical link: two rows written atomically on successful ingest: a `payload_export_records` row linking the vendor's new source record to the studio's canonical asset, and a `replicated_assets` stub so the record is immediately queryable via ArtHound.

Ingest template: a vendor's saved default field mapping for payloads from a specific studio. Auto-saved after each successful ingest. Keyed on `vendor_id + studio_id`, not on the dispatch, so it persists across multiple dispatch cycles.

Drift: a mismatch between the vendor's saved ingest template and the current payload's field schema (new fields added, old fields removed by the studio since the template was last saved). Surfaced to the vendor as a resolution step before the mapping UI is shown.

Revocation: a studio action that immediately blocks all future vendor access to a dispatch. The vendor's already-created source tool record is not deleted (ArtHound does not own the vendor's Jira or Airtable), but no further canonical lookups can be performed against the revoked dispatch.

Data Flow

Studio side

```
Studio selects assets + vendor (+ optional template)
→ POST /api/payloads/dispatch-bulk
    └─ For each asset: INSERT payload_dispatches (frozen snapshot, SHA-256 token hash)
    └─ _log(dispatch_id, "dispatched")
    └─ enqueue_attachment_copy(dispatch_id) → attachment_copy_jobs
```

Each dispatch is independent. If 20 assets are selected, 20 rows are created. Batch failures are silent at the per-asset level; the response returns `{dispatched: N, dispatch_ids: [ ... ]}` only.

The studio can view the outbox via `GET /api/payloads/outbox`, which includes view counts from `payload_access_log` and the `ingested_at` / `failed_at` state for each dispatch.

Attachment pipeline

After dispatch, a background worker polled by `main.py` drains `attachment_copy_jobs`:

```
attachment_copy_jobs (pending)
→ lib/attachments.copy_payload_attachments()
    └─ Walk payload_data fields for attachment-type values
    └─ Download each blob from studio's source tool (Airtable / Jira)
    └─ SHA-256 hash the bytes → content-addressed path
    └─ Upload to Supabase Storage at attachments/sha256/{hash} (skip if already exists)
    └─ INSERT attachment_refs(content_hash, dispatch_id)
```

Blobs are deduplicated across dispatches by content hash. The same attachment shared across multiple dispatch cycles is stored once. Vendors access attachments via signed Supabase Storage URLs, they never connect to the studio's original Airtable or Jira environment.

Vendor side

```
GET /api/payloads/vendor-inbox
→ Lists non-expired, non-revoked dispatches for the authenticated vendor
→ Sets received_at on first access

GET /api/payloads/{id}/ingest-schema
→ Returns frozen payload schema merged with vendor's source tool schema
→ Loads saved ingest template for this studio (if any)
→ Computes drift: new fields vs removed fields vs template
→ Returns drift resolution items if mismatch detected

POST /api/payloads/{id}/mapping
→ Saves vendor's field mapping to payload_field_mappings

POST /api/payloads/{id}/ingest
→ See ingest flow below
```

Ingest flow

```
POST /api/payloads/{id}/ingest
1. Validate: dispatch not expired, not revoked
2. Check payload_export_records for prior ingest of this canonical asset by this vendor
   └─ If found: reuse existing source_record_id (no duplicate creation), mark ingested_at
3. Build target_fields from field mapping + type coercion (ADF for Jira, multipleSelect for Jira)
4. connector.create_issue() / connector.create_record() in vendor's source tool
5. PATCH ingested_source_record_id onto payload_field_mappings
6. _write_canonical_link() - 3-attempt retry with exponential backoff:
   └─ POST payload_export_records (dispatch → canonical_asset → vendor_tool_record_id)
   └─ POST replicated_assets stub (idempotent; resolution=ignore-duplicates)
7a. Success: PATCH ingested_at → terminal success; auto-save ingest template for this studio
7b. Failure: PATCH failed_at + failure_reason; POST failed_ingests → terminal retryable state
```

`ingested_at` is not set until both the external write and the canonical link write succeed. The system never reports partial success as complete.

Retry (canonical link failure)

If the external record was created but the canonical link write failed after all retries:

```
POST /api/payloads/{id}/retry-canonical
1. Look up failed_ingests row for dispatch
2. Re-run _write_canonical_link() using stored source_record_id
   (does not touch the vendor's source tool again)
3. On success: PATCH ingested_at, clear failed_at, mark failed_ingests.resolved_at
```

The `failed_ingests` table carries `export_ok` / `replicated_ok` booleans so operator-level remediation targets exactly the table(s) that failed.

Security

Token design

The one-time plaintext token is returned to the studio at dispatch time and never persisted.

`payload_dispatches` stores only the SHA-256 hex (`token_hash`). In practice, vendors authenticate routes with their Supabase JWT and the route checks `recipient_vendor_id` ; the token provides an additional layer of revocation control.

Row Level Security

All payload tables have RLS enabled. Writes to `payload_access_log` are blocked at the policy level, only the service-role `_log()` helper can write audit rows.

Table	Studio	Vendor
<code>payload_templates</code>	All ops, own <code>studio_id</code>	No access
<code>payload_dispatches</code>	All ops where <code>sender_studio_id = own studio</code>	SELECT only where <code>recipient_vendor_id = own vendor</code>
<code>payload_field_mappings</code>	SELECT via FK chain through dispatches	SELECT / INSERT / UPDATE scoped to own <code>vendor_id</code>
<code>payload_access_log</code>	SELECT via FK chain through dispatches	SELECT via FK chain through dispatches
<code>failed_ingests</code>	No direct access	No direct access
<code>attachment_copy_jobs</code>	No direct access	No direct access

Cross-tenant isolation

- The studio never sees the vendor's internal source tool schema or mapping choices
- The vendor never sees the studio's Supabase data, only the frozen `payload_data` blob
- Attachments travel through ArtHound Storage with signed URLs; the vendor has zero connection to the studio's original Airtable or Jira environment
- Data never crosses org boundaries without an active `studio_vendor_links` record

Expiry and revocation

Every ingest checks `revoked_at IS NULL AND expires_at > now()` . Revoking a dispatch blocks all future access immediately, including attachment URL generation. The vendor's already-created source records are not deleted.

Data Hosting

Data	Hosted at	Controlled by
Studio's live source data	Studio's source tool (Airtable / Jira)	Studio
Canonical asset identity	ArtHound Supabase (<code>canonical_assets</code>)	ArtHound (neutral)
Frozen payload snapshot	ArtHound Supabase (<code>payload_dispatches.payload_data</code>)	Studio (immutable after dispatch)
Attachment blobs	ArtHound Supabase Storage (<code>attachments/sha256/</code>)	ArtHound (content-addressed)
Vendor field mappings	ArtHound Supabase (<code>payload_field_mappings</code>)	Vendor
Vendor-created record	Vendor's source tool (Jira / Airtable)	Vendor
Canonical link	ArtHound Supabase (<code>payload_export_records</code>)	ArtHound (written on ingest)
Audit trail	ArtHound Supabase (<code>payload_access_log</code>)	ArtHound (append-only, service role only)

ArtHound acts as the neutral clearing house. It holds the snapshot and the linkage records but does not own the studio's live data or the vendor-created records.

Inline Content Viewing

Vendors can review the payload field data inline in the ingest UI before committing. The payload schema is self-describing (`{key, label, type}` per field) and each field's value is visible in the mapping table alongside the vendor's target field picker.

Attachments are viewable via signed Supabase Storage URLs once the copy job completes. A PDF viewer (`pdf.js`) handles PDF attachments; images are displayed inline. Video attachments are not currently supported for inline playback.

There is no read-only "preview" mode that lets a vendor see the full asset record before entering the mapping flow, data is visible only in the context of the mapping table itself.

Known Gaps

Attachment lifecycle: blob purge on revocation or expiry is not enforced. Revoking a dispatch blocks API access but the bytes may remain in Storage until a manual purge. Stale copy handling and per-

studio quotas are an open design item.

No per-asset failure detail on bulk dispatch: if one asset in a bulk dispatch fails during snapshot creation, the rest succeed silently. The response returns only the count and IDs of successful dispatches.

No pre-dispatch preview: studios can build templates using a sample of 20 recent assets but there is no "preview what the vendor will see" confirmation step before a bulk dispatch is sent. A mis-configured template has no recovery path short of revocation.

No re-dispatch diff: if a studio re-dispatches an asset after a major revision, the vendor sees a new payload with no indication of what changed since the prior dispatch.

`payload_access_log` has no `actor_vendor_id`, vendor-side events (viewed, mapped, ingested) are logged without a vendor actor reference. Forensic tracing on the vendor side of the audit trail is limited to the dispatch ID.

Vendor-to-studio payload direction: the current system is strictly one-way (studio → vendor). Symmetric vendor → studio payloads (durations, deliverables) are a planned future capability.